

Corrigé du Partiel

VERSION CORRIGÉE — À NE CONSULTER QU'APRÈS AVOIR TRAITÉ LE SUJET

Arbres binaires — Expressions arithmétiques — Heapsort — ABR — LLRBT

PARTIE A — THÉORIE (4 PTS)

1.a Définition inductive + relation nœuds internes / externes

1 pt

✓ Un **arbre binaire** est défini inductivement :

- **Base** : l'arbre vide (représenté par NULL) est un arbre binaire.
- **Induction** : si l et r sont des arbres binaires et v une étiquette, alors le triplet (l, v, r) est un arbre binaire (un nœud interne avec fils gauche l et fils droit r).

Relation fondamentale : un arbre binaire de taille n (n nœuds internes) possède exactement **$n + 1$ nœuds externes**.

Preuve par induction : arbre vide $\rightarrow 0$ interne, 1 externe ✓. Si la propriété est vraie pour l (n_1 , n_1+1) et r (n_2 , n_2+1), l'arbre obtenu a n_1+n_2+1 internes et $(n_1+1)+(n_2+1) = n_1+n_2+2 = (n_1+n_2+1)+1$ externes ✓

1.b Hauteur minimale et maximale pour $n \geq 1$

1 pt

✓ **Hauteur minimale** = $\lceil \log_2 n \rceil$ — atteinte par un arbre *complet* (toutes les feuilles au même niveau ou au niveau inférieur), aussi appelé arbre parfaitement équilibré.

Hauteur maximale = $n - 1$ — atteinte par une *liste chaînée* (chaque nœud n'a qu'un seul fils).

Exemple pour $n = 4$:

Hauteur min = 2 (arbre complet)

Hauteur max = 3 (liste)

```

A
 / \
B   C
 /
D

```

```

A
 \
B
 \
C
 \
D

```

1.c Arbres distincts de taille 3

1 pt

✓ $C_3 = 5$ arbres distincts :

```

o
 / \
o   o
 |
o

```

(1)

```

o
 / \
o   o
 |
o

```

(2)

```

o
 | \
o   o
 |
o

```

(3)

```

o
 \
o
 | \
o   o

```

(4)

```

o
 /
o
 | \
o   o

```

(5)

⚠ Les étiquettes ne comptent pas pour distinguer les formes — seule la structure topologique est considérée.

1.d Récurrence des nombres de Catalan, calcul de C_4

1 pt

✓ **Réurrence :** $C_0 = 1$, et pour $n \geq 0$:

$$C_{n+1} = \sum_{k=0}^n C_k \cdot C_{n-k}$$

(la racine sépare un sous-arbre gauche de taille k et un droit de taille $n-k$, pour k de 0 à n)

Calcul : $C_0=1, C_1=1, C_2=2, C_3=5$

$$C_4 = C_0 \cdot C_3 + C_1 \cdot C_2 + C_2 \cdot C_1 + C_3 \cdot C_0 = 1 \times 5 + 1 \times 2 + 2 \times 1 + 5 \times 1 = \mathbf{14}$$

PARTIE B — PROGRAMMATION C (4 PTS)

1.e int sum_binary_tree(link h)

2 pts

✓

```
int sum_binary_tree(link h) {
    if (h == NULL)
        return 0;
    return h->label
        + sum_binary_tree(h->left)
        + sum_binary_tree(h->right);
}
```

1 pt : cas de base correct. 1 pt : appels récursifs gauche + droite + étiquette.

1.f int height_binary_tree(link h)

2 pts

✓

```
int height_binary_tree(link h) {
    if (h == NULL)
        return -1;
    int hl = height_binary_tree(h->left);
    int hr = height_binary_tree(h->right);
    return 1 + (hl > hr ? hl : hr);
}
```

1 pt : cas de base -1 . 1 pt : formule $1 + \max(hg, hd)$. Accepté aussi : utiliser une fonction max séparée.

PARTIE A — COMPRÉHENSION (3 PTS)

2.a Arbre et valeur de $* + 3\ 4 - 8\ 2$

1 pt

```
*
 / \
+ -
 / \ / \
3 4 8 2
```

Valeur : $(3+4) \times (8-2) = 7 \times 6 = 42$

2.b Expression préfixe et infixe de l'arbre donné

1 pt

Notation préfixe : $+ * 2\ 3\ 5$ **Notation infixe (avec parenthèses) :** $((2*3)+5)$ **Valeur :** $(2 \times 3) + 5 = 11$ 2.c Pourquoi `const char**` et non `const char*` ?

1 pt

Avec `char*`, le paramètre est passé **par valeur** : chaque appel récursif reçoit une *copie* du pointeur. Avancer ce pointeur n'affecte pas les autres appels — le curseur ne progresse pas globalement.

Avec `char**`, on passe l'**adresse du pointeur**. Tous les appels récursifs partagent le *même* curseur :

- `**s` → lit le caractère courant
- `(*s)++` → avance le curseur, visible par tous les appels

Sans cela, chaque appel recommencerait à lire depuis la même position, causant une boucle infinie ou un résultat faux.

PARTIE B — PROGRAMMATION C (6 PTS)

2.d `parse_aux` et `parse_expr`

3 pts

```

static link parse_aux(const char **s) {
    /* Ignorer les espaces */
    while (**s == ' ') (*s)++;

    char c = **s;
    (*s)++;                                /* avancer d'un caractère */

    if (c == '+' || c == '-' || c == '*' || c == '/') {
        link left = parse_aux(s); /* parser le fils gauche */
        link right = parse_aux(s); /* parser le fils droit */
        return cons_binary_tree(c, left, right);
    } else {
        /* c est un chiffre : créer une feuille */
        return cons_binary_tree(c, NULL, NULL);
    }
}

link parse_expr(const char *s) {
    return parse_aux(&s);                /* passe l'ADRESSE du pointeur */
}

```

1 pt : `&s` dans `parse_expr`. 1 pt : `(*s)++` correct. 1 pt : récursion gauche puis droite pour les opérateurs.

2.e int eval_tree(link h)

3 pts

```

int eval_tree(link h) {
    /* Cas de base : feuille (chiffre) */
    if (h->left == NULL && h->right == NULL)
        return h->label - '0';          /* convertir char -> int */

    /* Cas récursif : nœud opérateur */
    int l = eval_tree(h->left);
    int r = eval_tree(h->right);

    switch (h->label) {
        case '+': return l + r;
        case '-': return l - r;
        case '*': return l * r;
        case '/': return l / r;          /* division entière */
    }
    return -1;                          /* ne devrait pas arriver */
}

```

1 pt : cas de base (- '0'). 1 pt : appels récursifs l et r. 1 pt : les 4 opérateurs correctement branchés.

PARTIE A — THÉORIE (4 PTS)**3.a** Propriété du tas-max + formules

1 pt

- ✓ **Propriété tas-max** : pour tout nœud d'indice i , $a[i] \geq a[2i]$ et $a[i] \geq a[2i+1]$ (l'étiquette de chaque nœud est supérieure ou égale à celles de ses fils).

Formules (base 1) :

- Fils gauche de i : $2i$
- Fils droit de i : $2i + 1$
- Parent de i : $\lfloor i/2 \rfloor$

Structure : un arbre binaire **complet** (toutes les feuilles au même niveau ou au niveau inférieur, complété de gauche à droite).

3.b Simulation de `buildHeap` sur `[4, 10, 3, 5, 1, 2, 6]`

2 pts

- ✓ La dernière feuille interne est à l'indice $\lfloor 7/2 \rfloor = 3$. On appelle `fixHeap` de $i=3$ à $i=1$.

Tableau initial :

[_, 4, 10, 3, 5, 1, 2, 6]
1 2 3 4 5 6 7

$i=3$: $a[3]=3$, fils gauche $a[6]=2$, fils droit $a[7]=6$
 $\max \text{ fils} = a[7]=6 > a[3]=3 \rightarrow \text{swap}(3,7)$
 [_, 4, 10, 6, 5, 1, 2, 3]

$i=2$: $a[2]=10$, fils $a[4]=5$, $a[5]=1$
 $\max \text{ fils} = a[4]=5 < a[2]=10 \rightarrow \text{rien}$
 [_, 4, 10, 6, 5, 1, 2, 3]

$i=1$: $a[1]=4$, fils $a[2]=10$, $a[3]=6$
 $\max \text{ fils} = a[2]=10 > a[1]=4 \rightarrow \text{swap}(1,2)$
 [_, 10, 4, 6, 5, 1, 2, 3]
 Descendre $a[2]=4$: fils $a[4]=5$, $a[5]=1$
 $\max \text{ fils} = a[4]=5 > a[2]=4 \rightarrow \text{swap}(2,4)$
 [_, 10, 5, 6, 4, 1, 2, 3]
 Descendre $a[4]=4$: fils $a[8]$, $a[9] \rightarrow \text{n'existent pas} \rightarrow \text{stop}$

Résultat final (tas-max) : [_, 10, 5, 6, 4, 1, 2, 3]

△ Vérifier : $10 \geq 5$ ✓, $10 \geq 6$ ✓, $5 \geq 4$ ✓, $5 \geq 1$ ✓, $6 \geq 2$ ✓, $6 \geq 3$ ✓ $\rightarrow$ c'est bien un tas-max.

3.c Pourquoi `buildHeap` est $O(n)$ et non $O(n \log n)$?

1 pt

- ✓ On appelle `fixHeap` sur les $\lfloor n/2 \rfloor$ nœuds internes, de bas en haut. Mais tous ces appels ne font pas $O(\log n)$ travail chacun : les nœuds proches des feuilles descendent très peu.

Analyse précise : il y a au plus $\lfloor n/2^{(h+1)} \rfloor$ nœuds à hauteur h , et `fixHeap` y coûte $O(h)$. La somme vaut :

$$\sum_{h=0}^{\lfloor \log n \rfloor} h \cdot n/2^h = O(n \cdot \sum h/2^h) = O(n \cdot 2) = \mathbf{O(n)}$$

car la série $\sum h/2^h$ converge vers 2.

PARTIE B — PROGRAMMATION C (4 PTS)**3.d** `void fixHeap(int *a, int l, int r, int i)`

4 pts

```

void swap(int *a, int i, int j) {
    int tmp = a[i]; a[i] = a[j]; a[j] = tmp;
}

void fixHeap(int *a, int l, int r, int i) {
    while (1) {
        /* Calculer les indices des fils dans le sous-tableau a[l..r] */
        int left  = l + 2*(i - l) + 1;
        int right = l + 2*(i - l) + 2;

        /* Trouver le plus grand parmi i, fils gauche, fils droit */
        int largest = i;
        if (left  <= r && a[left]  > a[largest]) largest = left;
        if (right <= r && a[right] > a[largest]) largest = right;

        /* Si i est déjà le plus grand, le tas est restauré */
        if (largest == i) break;

        /* Sinon, échanger et continuer la descente */
        swap(a, i, largest);
        i = largest;
    }
}

```

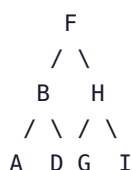
1 pt : formule fils gauche/droit correcte. 1 pt : calcul du plus grand des 3. 1 pt : condition d'arrêt correcte. 1 pt : boucle itérative (accepté aussi : récursif).

PARTIE A — THÉORIE (3 PTS)

4.a Insertion de F, B, H, A, D, G, I + parcours infixe

1 pt

Insertion dans l'ordre F, B, H, A, D, G, I :



Parcours infixe : A B D F G H I (*ordre alphabétique — propriété ABR*)

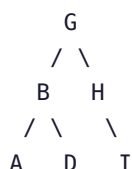
4.b Suppression de F (la racine) via `join_BST`

1 pt

Étapes :

- On identifie L = sous-arbre gauche de $F = \{B, A, D\}$ et R = sous-arbre droit de $F = \{H, G, I\}$
- On libère le nœud F (`free(h)`)
- On appelle `join_BST(L, R)` : on amène le minimum de R à la racine via `partition_BST(R, 0)`
- Le minimum de R est G (rang 0 dans $\{H, G, I\}$). Après partition, G est à la racine de R .
- $G.\text{left} \leftarrow L$

Arbre résultant :

4.c Rôle de `partition_BST(h, k)`

1 pt

`partition_BST(h, k)` amène le nœud de **rang k** (le $(k+1)$ -ème plus petit) à la racine, en restructurant l'arbre par des rotations.

Briques : `size_BST` (pour calculer le rang de la racine courante) + `rotate_left` / `rotate_right` (pour faire monter le bon nœud).

Complexité : $O(n)$ dans le pire cas (arbre dégénéré), $O(\log n)$ en moyenne. La propriété ABR est conservée.

PARTIE B — PROGRAMMATION C (6 PTS)

4.d `link partition_BST(link h, int k)`

3 pts


```

link partition_BST(link h, int k) {
    int t = size_BST(h->left);    /* rang de la racine courante */

    if (k < t) {
        /* Le nœud voulu est dans le sous-arbre gauche */
        h->left = partition_BST(h->left, k);
        h = rotate_right(h);      /* faire monter le fils gauche */
    } else if (k > t) {
        /* Le nœud voulu est dans le sous-arbre droit */
        h->right = partition_BST(h->right, k - t - 1);
        h = rotate_left(h);       /* faire monter le fils droit */
    }
    /* k == t : la racine courante a déjà rang k → rien à faire */
    return h;
}

```

1 pt : calcul de $t = \text{size_BST}(h \rightarrow \text{left})$. 1 pt : récursion correcte sur gauche/droite. 1 pt : rotations dans le bon sens avec $k-t-1$ pour la droite.

4.e link join_BST(link L, link R)

3 pts

```

link join_BST(link L, link R) {
    if (L == NULL) return R;      /* cas de base : L vide */
    if (R == NULL) return L;      /* cas de base : R vide */

    /* Amener le minimum de R (rang 0) à la racine de R */
    R = partition_BST(R, 0);

    /* Accrocher L comme fils gauche du nouveau minimum */
    R->left = L;

    return R;
    /* Pas d'allocation : on réutilise les nœuds existants */
}

```

1 pt : cas de base L ou R NULL. 1 pt : `partition_BST(R, 0)` amène le min. 1 pt : `R->left = L` et retour sans free ni malloc.

⚠ Erreur fréquente : appeler `partition_BST(L, size_BST(L)-1)` au lieu de `partition_BST(R, 0)`. Les deux fonctionnent mais la version avec R est plus simple.

PARTIE A — THÉORIE (3 PTS)**5.a** Les 5 propriétés du LLRBT + borne sur la hauteur

2 pts

✓

- **P1** : La propriété ABR est vérifiée (clés gauche < racine ≤ clés droite).
- **P2** : Un lien rouge penche *toujours à gauche* : il n'y a jamais de lien rouge allant vers un fils droit.
- **P3** : Il n'y a jamais deux liens rouges consécutifs (pas de double-rouge sur un même chemin).
- **P4** : La racine est toujours noire.
- **P5** : Tous les chemins de la racine vers les feuilles contiennent le même nombre de liens noirs (propriété de hauteur noire parfaite).

Borne sur la hauteur : la hauteur noire h_n donne $2^{h_n} \leq n+1$, donc $h_n \leq \log_2(n+1)$. La hauteur totale est au plus $2 \times h_n$ (alternance rouge-noir), donc :

$$\text{hauteur} \leq 2 \cdot \log_2(n+1)$$

5.b Correspondance LLRBT ↔ arbre 2-3

1 pt

✓

Un LLRBT encode un **arbre 2-3** (sans 4-nœuds) :

- Un **2-nœud** (une clé, deux fils) correspond à un nœud noir sans fils gauche rouge.
- Un **3-nœud** (deux clés $a < b$, trois fils) est encodé par un nœud noir b dont le fils gauche est un nœud rouge a . Les trois sous-arbres de a et b deviennent les trois fils du 3-nœud.

Le lien rouge représente une "fusion" de deux nœuds BST en un seul nœud 2-3. Incliner à gauche impose un encodage canonique unique.

PARTIE B — SIMULATION : INSERTION DE S, E, A, R, C, H (3 PTS)**5.c** Simulation complète

3 pts



Convention : (R) = nœud rouge, (B) = nœud noir. On insère toujours en rouge ; la racine est rendue noire après chaque insertion.

— Insérer S —————
S(B) ← racine rendue noire. Aucune correction.

— Insérer E —————
S(B) E est < S → fils gauche rouge
/
E(R)
✓ Pas de correction nécessaire.

— Insérer A —————
S(B) A est < E → fils gauche de E
/
E(R)
/
A(R) ← DOUBLE ROUGE à gauche !

[rotate_right(S)] :
E(R)
/ \
A(R) S(B) ← rouge à gauche consécutif résolu mais E(R) remonte

E est maintenant rouge ET a deux fils rouges (A rouge, S rouge) :
[color_flip(E)] :
E(B)
/ \
A(R) S(R) puis racine → E(B) ✓

— Insérer R —————
E(B) R est entre E et S → R va à droite de E, gauche de S
/ \
A(R) S(B)
/
R(R)
S a un fils gauche rouge R et un fils droit absent → OK (penche à gauche ✓)
Pas de correction.

— Insérer C —————
E(B)
/ \
A(R) S(B) C est entre A et E → fils droit de A
/
R(R)
A a un fils droit rouge C :
[rotate_left(A)] :
E(B)
/ \
C(R) S(B) C monte, A descend à gauche
/ /
A(R) R(R)

C a deux fils rouges A et... non, A est seul fils gauche.
C(R) avec fils gauche A(R) = double rouge à gauche !
[rotate_right(E)] :
C(R)
/ \
A(R) E(B)
\
S(B)
/
R(R)

C a deux fils rouges A et E :
[color_flip(C)] :

```

      C(B)
     /  \
A(B) E(R)
     \
      S(B)
     /
    R(R)
Racine → C(B) ✓

```

— Insérer H —————

```

      C(B)
     /  \
A(B) E(R)   H est entre E et S → fils gauche de S (H < S et H > R)
     \
      S(B)   H > R donc fils droit de R
     /
    R(R)
     \
    H(R) ← rouge à droite !

```

[rotate_left(R)] :

R remplace H, R passe à gauche de H :

```

      S(B)
     /
    H(R)
   /
  R(R) ← double rouge à gauche !

```

[rotate_right(S)] :

```

      H(R)
     /  \
    R(R) S(B)

```

H a deux fils rouges → [color_flip(H)] :

```

      H(B)
     /  \
    R(B) S(B)   H remonte dans l'arbre principal...

```

E a maintenant un fils droit H(B) rouge → non, H est noir.

E avait fils droit S(B) qui est devenu H(B)... OK.

Arbre final après S, E, A, R, C, H :

```

      C(B)
     /  \
A(B) E(R)
     \
      H(B)
     /  \
    R(B) S(B)

```

Vérification : parcours infixe = A C E H R S ✓ (ordre alphabétique)

1 pt : insertions S, E, A correctes avec corrections. 1 pt : insertions R, C correctes. 1 pt : insertion H correcte.

BARÈME RÉCAPITULATIF			
Exercice	Thème	Détail	Total
1	Arbres binaires	1+1+1+1 (théorie) + 2+2 (code)	8
2	Expressions préfixées	1+1+1 (compréhension) + 3+3 (code)	9
3	Heapsort	1+2+1 (théorie) + 4 (code)	8
4	ABR	1+1+1 (théorie) + 3+3 (code)	9
5	LLRBT	2+1 (théorie) + 3 (simulation)	6
TOTAL			40